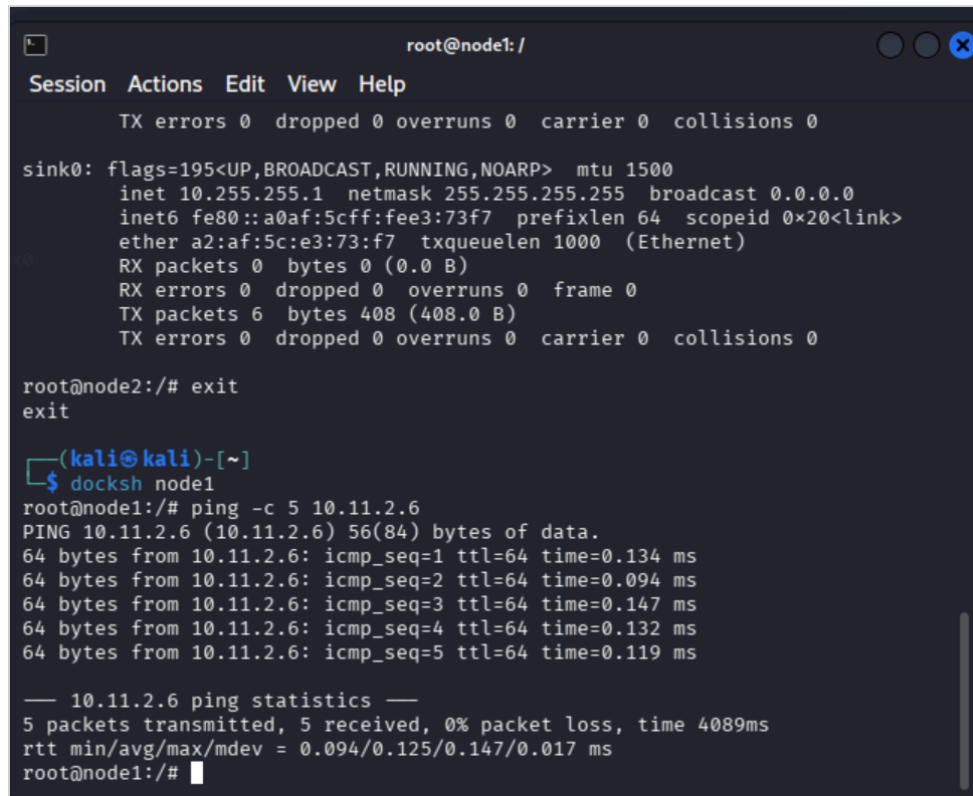


Lab 1: Packet Sniffing and Spoofing

Network Security (50.020) · Name: <Name> · Student ID: <StudentID>

Environment. One Kali VM running three Docker containers on an isolated network 10.11.2.0/24: node1 10.11.2.5 (MAC c2:ea:c3:7e:d6:be), node2 10.11.2.6 (MAC b6:8f:0b:f5:a6:a0), node3 10.11.2.7.

1. Generate Network Traffic



```
root@node1: /
Session Actions Edit View Help
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

sink0: flags=195<UP,BROADCAST,RUNNING,NOARP> mtu 1500
inet 10.255.255.1 netmask 255.255.255.255 broadcast 0.0.0.0
inet6 fe80::a0af:5cff:fee3:73f7 prefixlen 64 scopeid 0<link>
ether a2:af:5c:e3:73:f7 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 6 bytes 408 (408.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@node2:/# exit
exit

(kali@kali)-[~]
$ docksh node1
root@node1:/# ping -c 5 10.11.2.6
PING 10.11.2.6 (10.11.2.6) 56(84) bytes of data.
64 bytes from 10.11.2.6: icmp_seq=1 ttl=64 time=0.134 ms
64 bytes from 10.11.2.6: icmp_seq=2 ttl=64 time=0.094 ms
64 bytes from 10.11.2.6: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 10.11.2.6: icmp_seq=4 ttl=64 time=0.132 ms
64 bytes from 10.11.2.6: icmp_seq=5 ttl=64 time=0.119 ms

— 10.11.2.6 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4089ms
rtt min/avg/max/mdev = 0.094/0.125/0.147/0.017 ms
root@node1:/#
```

Figure 1. `ping -c 5 10.11.2.6` from node1: 5 sent, 5 received, 0% loss, average RTT 0.125 ms.

Q: What does a successful Ping confirm? Does it mean all services on node2 are accessible?

It confirms that node1 and node2 have IP-layer reachability: packets are routed in both directions, node2 is up, and ICMP Echo is not blocked. It does **not** prove that services are accessible. Ping tests only ICMP; TCP/UDP services may not be running, may listen only on localhost, or may be blocked by a firewall. The reverse is also possible: ICMP can be blocked while services remain reachable.

```

(kali㉿kali)-[~]
└─$ docksh node1
root@node1:/# telnet 10.11.2.6
Trying 10.11.2.6 ...
Connected to 10.11.2.6.
Escape character is '^]'.

Linux 6.19.14+kali-amd64 (node2) (pts/3)

node2 login: root
Password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.19.14+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
root@node2:~#

```

Figure 2. Telnet from node1 to node2 (root /toor), logged in to root@node2 .

```

Session Actions Edit View Help
root@node1:/# ssh kali@10.11.2.6
kali@10.11.2.6's password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.19.14+kali-amd64 x86_64)
[ OK ] /shared/visible from host + all 3 nodes
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
LAB READY
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Sep 28 07:23:24 2026 from 10.11.2.5
kali@node2:~$

```

Figure 3. SSH from node1 to node2 as kali , logged in to kali@node2 .

Q: What is the main security weakness of Telnet?

Telnet sends everything, including the username and password, in plaintext. It provides no encryption, integrity protection, or server authentication, so anyone on the path can capture credentials and commands, alter the session, or impersonate the server. SSH encrypts the session, authenticates the server with host keys, and protects integrity, which is why Telnet is unsuitable for modern remote administration.

2. Packet Sniffing and Traffic Analysis

2.1 tcpdump

On node2, `ip addr` shows that `eth0` (10.11.2.6/24) is the interface on the lab network. `lo` is loopback, and `sink0` is a dummy interface used for off-subnet routes.

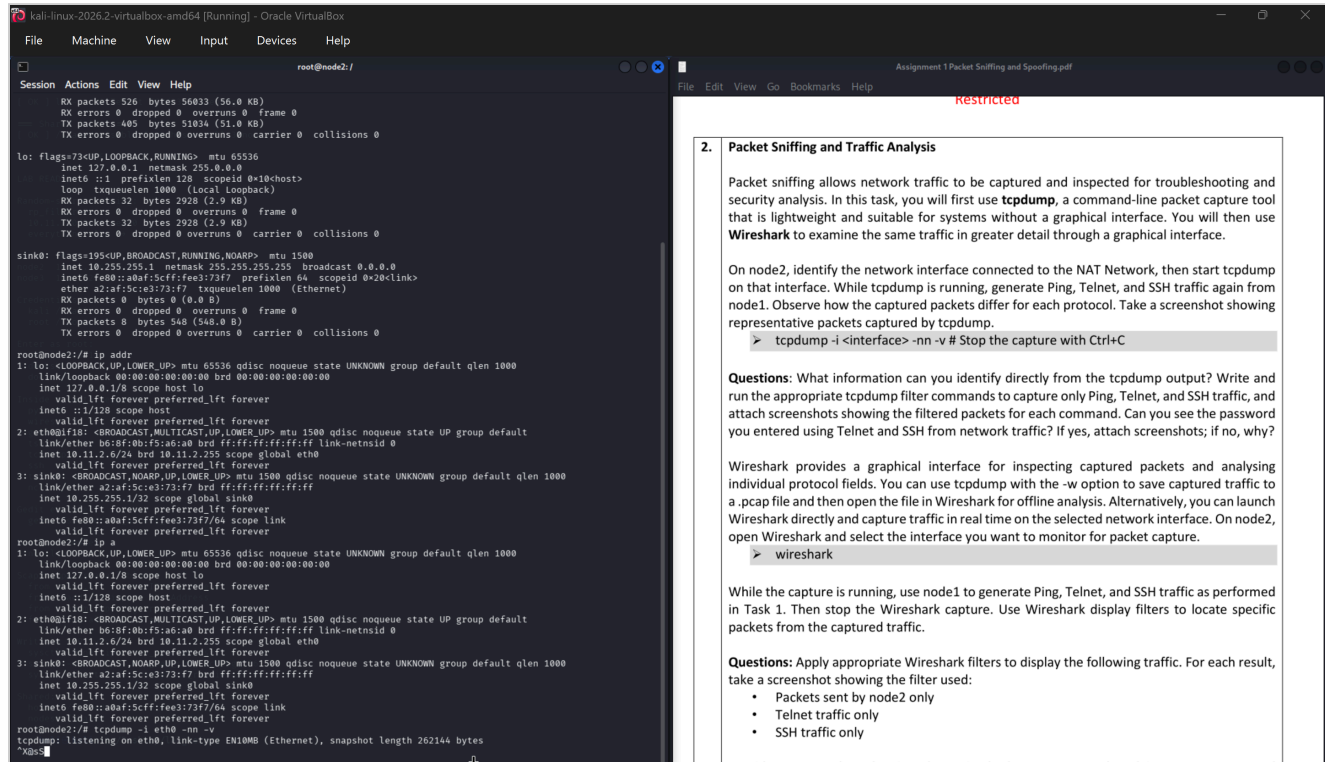


Figure 4. node2 interfaces and `tcpdump -i eth0 -nn -v` listening on `eth0`.

Q: What information can be identified from tcpdump output?

- Timestamp, source and destination IP addresses and ports, and protocol.
- IP header fields (`-v`): TOS, TTL, IP ID, flags (e.g. DF), protocol, and total length.
- TCP flags (SYN, ACK, PSH, FIN), sequence and acknowledgement numbers, window, options, and payload length.
- ICMP type (echo request/reply), identifier, and sequence number.
- Payload contents with `-A` (ASCII) or `-x` (hex and ASCII).

Filter commands:

```
tcpdump -i eth0 -nn -v icmp # Ping only
tcpdump -i eth0 -nn -v -A 'tcp port 23' # Telnet only (ASCII payload)
tcpdump -i eth0 -nn -v 'tcp port 22' # SSH only
```

Q: Can the Telnet and SSH passwords be seen in the traffic?

Telnet: yes. Each keystroke is sent in cleartext, typically one character per packet, so `-A` reveals the password character by character (see Figure 8). **SSH: no.** After the version exchange and Diffie-Hellman key exchange (Figure 7), all traffic, including authentication, is encrypted. Only lengths and timing are visible.

2.2 Wireshark

Requirement	Display filter	Result (of 212 captured packets)
Packets sent by node2 only	<code>ip.src == 10.11.2.6</code>	94 displayed
Telnet traffic only	<code>telnet (or tcp.port == 23)</code>	44 displayed
SSH traffic only	<code>ssh (or tcp.port == 22)</code>	91 displayed

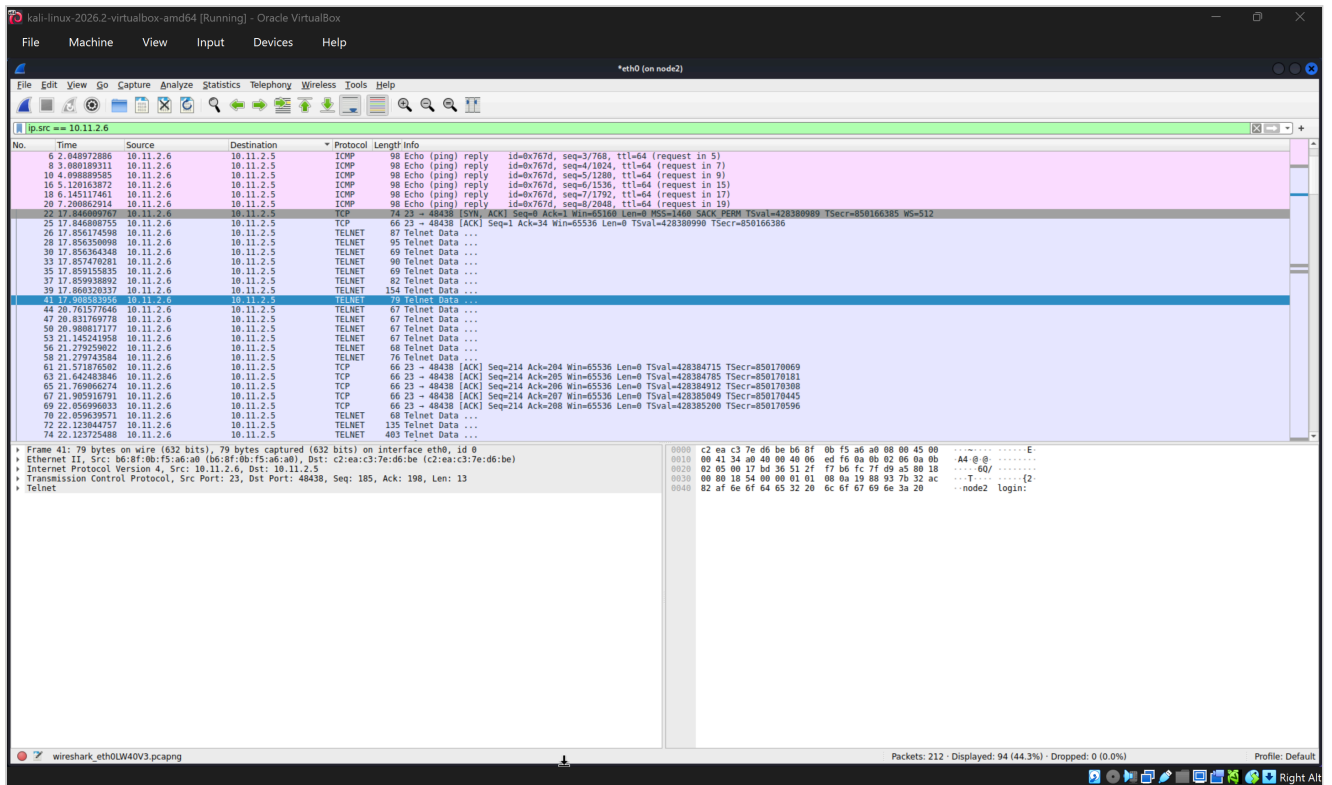


Figure 5. `ip.src == 10.11.2.6` : only node2's ICMP replies, TCP ACKs, and Telnet data.

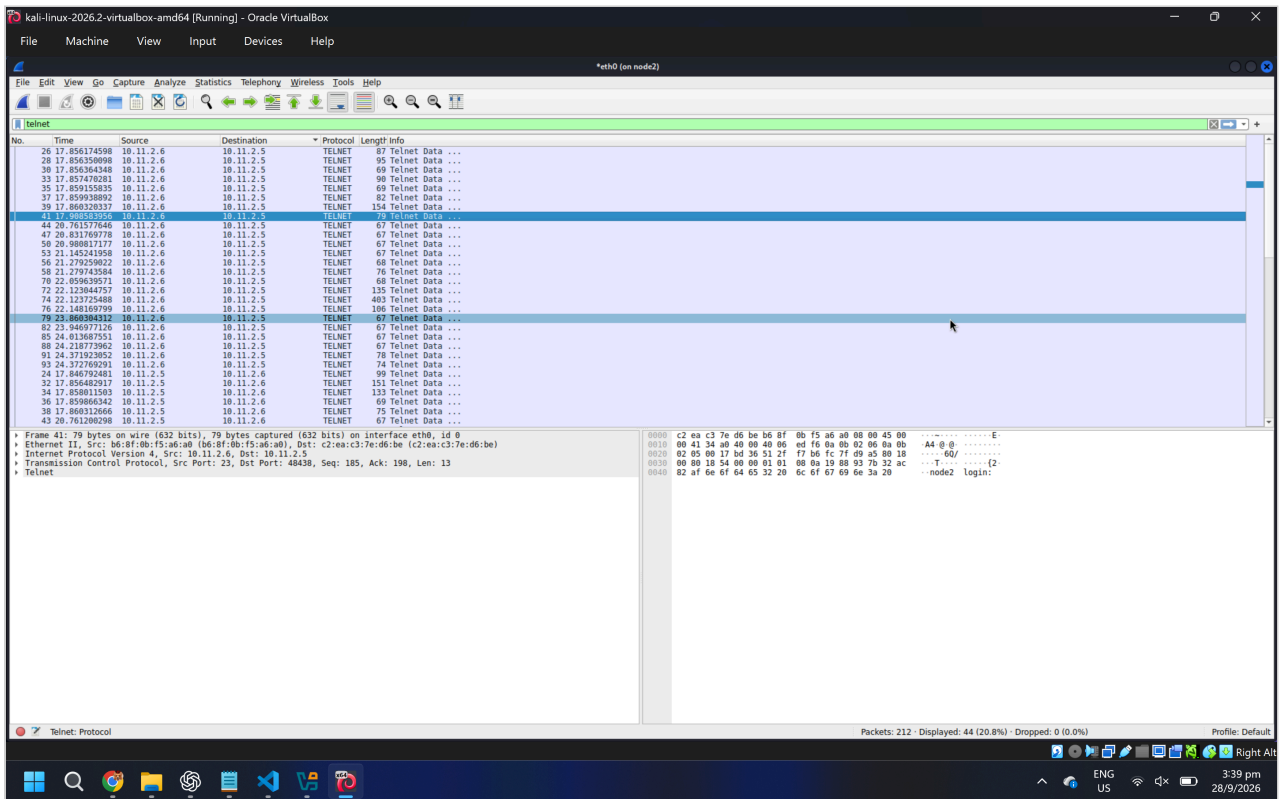


Figure 6. `telnet` filter: Telnet data between node1 and node2 on TCP port 23.

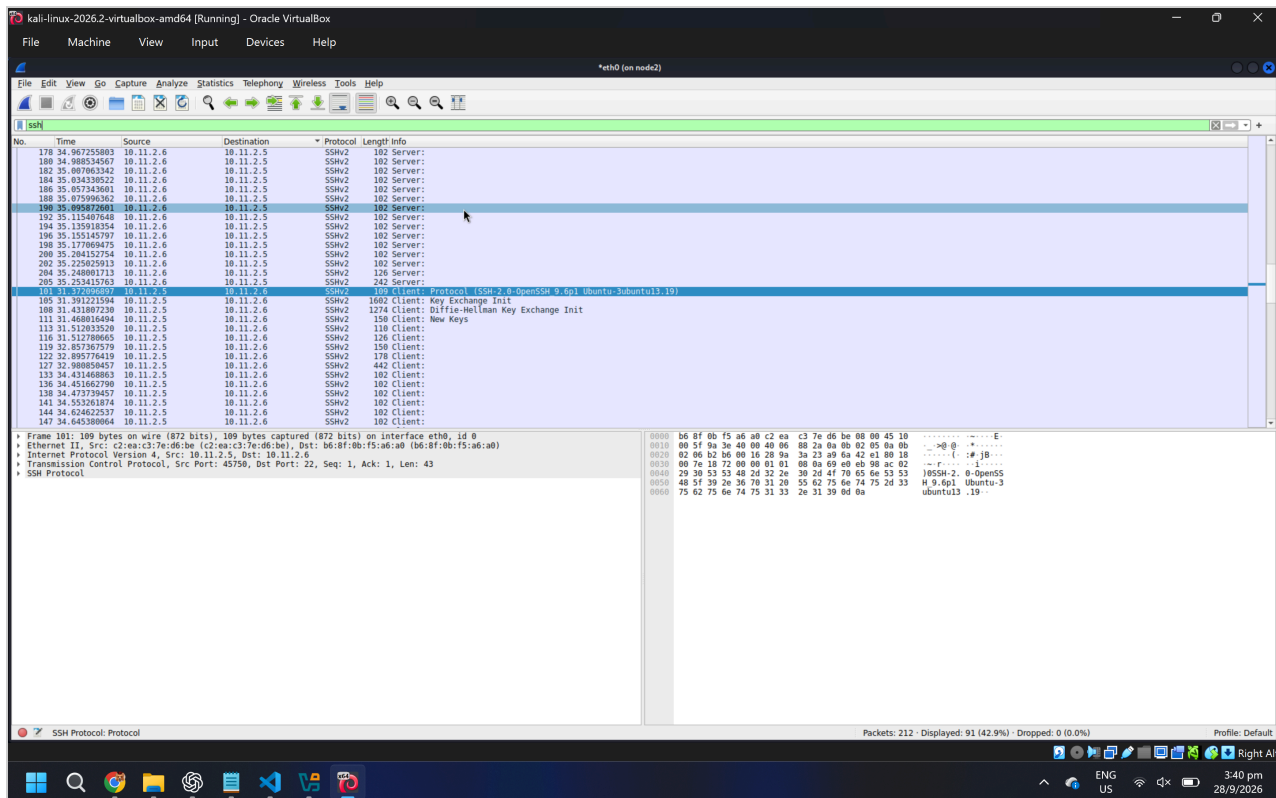


Figure 7. ssh filter: protocol banner, key exchange, and encrypted packets (Info shows only "Client:"/"Server:").

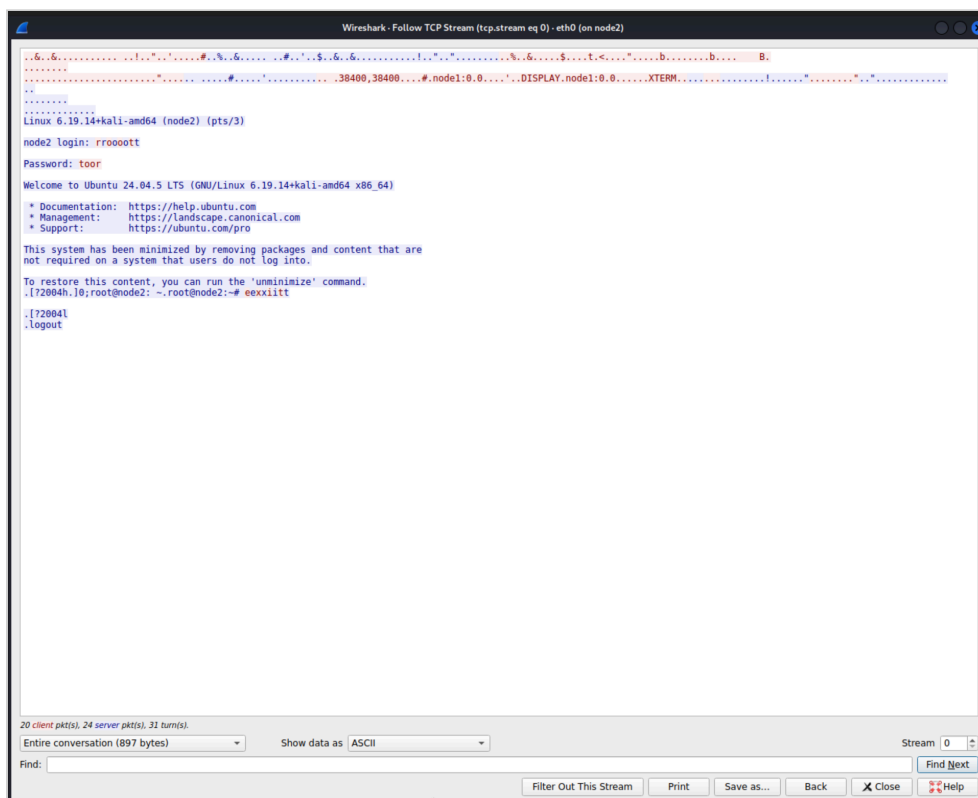


Figure 8. Follow TCP Stream of the Telnet session: username `root` and password `toor` in plaintext.

The username appears as `rroooooott` because node2 echoes each typed character: red is client-to-server and blue is server-to-client. The password `toor` appears once because the server does not echo it, but the client still sends it unencrypted.

Q: Why is Telnet a greater risk than SSH?

A passive sniffer on the same network can recover Telnet credentials and every command and response, as Figure 8 shows, and then reuse the credentials or hijack the session. SSH exposes only an encrypted stream. Its host-key verification also prevents an attacker from silently impersonating the server.

3. Source-IP Spoofing

`spooof_icmp.py` (run on node1) builds one ICMP Echo Request with a forged source address from the documentation range `198.51.100.0/24` (RFC 5737):

```
from scapy.all import ICMP, IP, send

packet = (IP(src="198.51.100.10", dst="10.11.2.6")
          / ICMP(type="echo-request", id=0x5002, seq=1)
          / b"Network Security Lab 1")
send(packet, iface="eth0", verbose=False)
```

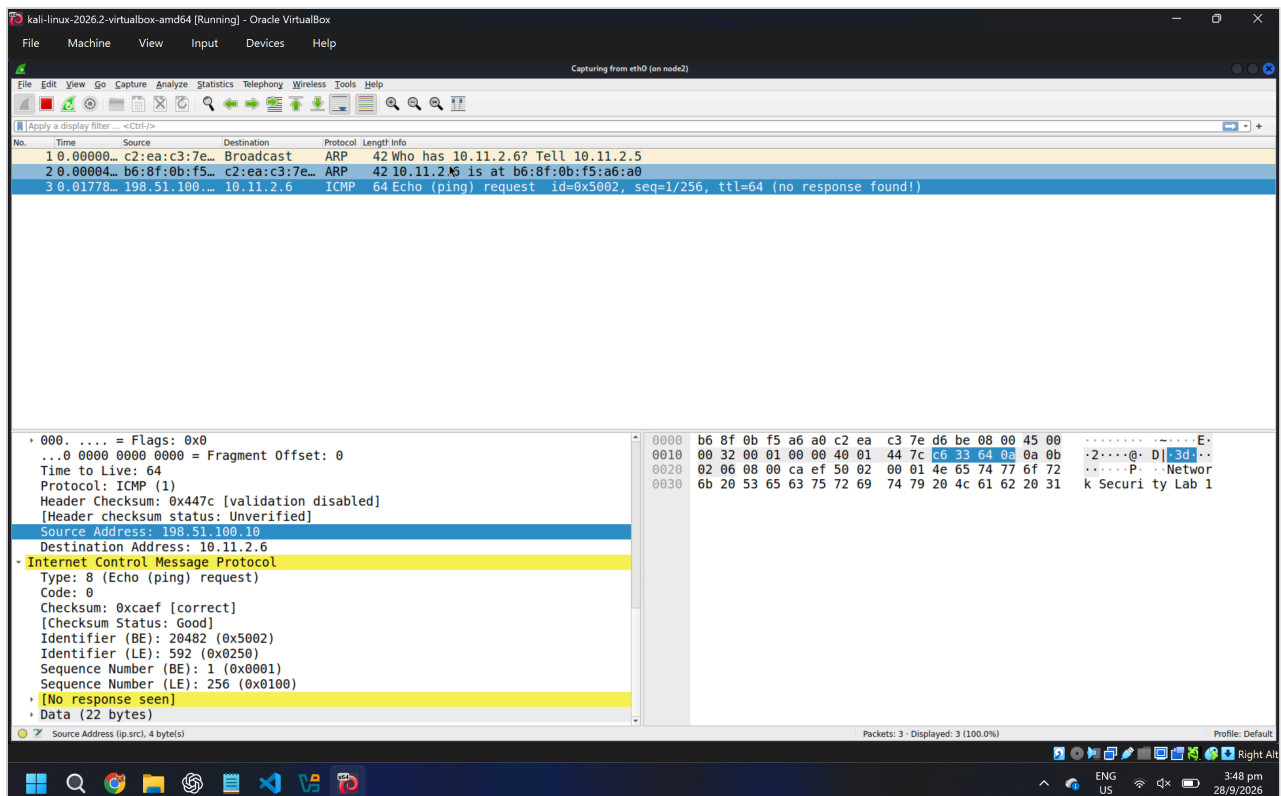


Figure 9. Wireshark on node2: Echo Request with source `198.51.100.10`, ID `0x5002`, seq 1, "no response found".

Q: What source IP does node2 observe? Is it node1's real address?

node2 sees `198.51.100.10`, not node1's real address `10.11.2.5`. The source IP field is chosen by the sender and is not authenticated, so a single packet's source address cannot be trusted as proof of origin.

Q: Does node1 receive the reply? Where would it be sent?

No. node2 addresses its Echo Reply to the packet's source, `198.51.100.10`. That address is outside `10.11.2.0/24`, so node2 routes the reply through its default route, which in this lab is the `sink0` dummy interface, and the reply is discarded. No reply appears on `eth0`. On a real network, the reply would go toward the forged address, not the real sender. Spoofing is therefore blind: the attacker cannot see responses. This is also how reflection attacks direct replies at a victim.

Q: Compare normal and spoofed ICMP packets. What else indicates suspicious traffic?

Field / behaviour	Normal ping (node1)	Spoofed packet
Source IP	<code>10.11.2.5</code> (local subnet)	<code>198.51.100.10</code> (off-subnet)

Source MAC	c2:ea:c3:7e:d6:be	c2:ea:c3:7e:d6:be , node1's MAC, which reveals the real sender
TTL	64	64: zero hops, which is impossible for a non-local source
Frame / payload	98 bytes; standard 56-byte ping payload with timestamp	64 bytes; custom 22-byte ASCII payload
ICMP ID / sequence	0x767d , sequence increments 1–5	Fixed 0x5002 , single seq 1
IP ID	Kernel-assigned	1 (Scapy default)
Reply	Echo Reply for every request	None ("no response found")
Preceding ARP	From 10.11.2.5	Also from 10.11.2.5 , not the claimed source

The strongest indicators are an IP/MAC mismatch, off-subnet sources arriving directly on the local segment, TTLs that are inconsistent with the claimed origin, unanswered requests, and non-standard payloads or header defaults.

4. AI-Based Traffic Analysis

Conclusion: An attack occurred. An unknown host, 10.0.2.8 (MAC 08:00:27:25:f3:ec), spoofed the trusted workstation 10.0.2.6 and injected 800 forged monitoring messages into node2's log service. The forged messages first reported a compromise and then flooded the server with false "all clear" status reports.

4.1 Features extracted and why

`analyze_incident.py` (Scapy) exports all 4,061 packets to `incident_packets.csv` with these fields:

Feature	Relevance
Source and destination MAC	Identifies the device that actually transmitted each frame on the LAN
Source and destination IP, UDP ports	Claimed identity and service (10.0.2.6:40000 → 10.0.2.7:5514, syslog-style)
ARP sender IP/MAC	Shows which IP each MAC legitimately owns
IP TTL and IP ID	Fingerprints the sending network stack
Timestamp	Timeline reconstruction and burst detection
Payload fields: host, seq, severity, event, status, user	What each message claims; sequence numbers should be unique and increasing

4.2 Use of AI and analysis techniques

- **LLM-assisted coding:** GitHub Copilot generated the Scapy PCAP-to-CSV extractor and aggregation queries. I verified the output against Wireshark and the raw counts.
- **LLM-assisted review:** The LLM read payload samples and summarized event distributions, flagging security-sensitive events that were absent from the baseline.
- **Rule-based anomaly detection:** Grouping by (source IP, source MAC) flagged one IP with two MACs. ARP data, sequence-number collisions, IP ID behaviour, and 30-second time bins then characterized the anomaly.

4.3 Evidence

Observation	Legitimate sender	Attacker
Claimed source IP	10.0.2.6	10.0.2.6
Source MAC	08:00:27:5a:87:bc	08:00:27:25:f3:ec
ARP identity of MAC	10.0.2.6 (17 ARP requests)	10.0.2.8 (2 ARP requests for node2)
Messages	3,200 across the entire capture	800, from 12:35:59 to 12:40:59
Sequence numbers	1–3200, all unique	602–2219, 713 unique; all reuse legitimate numbers, 698 with different content
IP ID	Increasing (43957, 43960, 44015, ...)	Random (39832, 55505, 8425, ...)
Event types	Heartbeat, CPU/memory, backup, auth, DNS, SSH running	Root/persistence events, then health and security "all clear"

The same claimed IP, ports, and host name (`workstation01`) are paired with a MAC that ARP ties to `10.0.2.8`. Duplicate sequence numbers with conflicting content and a different IP ID pattern confirm a second sender injecting into the stream. Throughout the attack, the real workstation continued sending routine telemetry and reported none of the attacker's events.

4.4 Reconstruction

1. **Discovery:** `10.0.2.8` resolved node2's MAC through ARP.
2. **Phase 1, about 12:36:00 (<30 s):** Forged ALERT/WARN messages claimed `workstation01` activity as `user=root: SUDO_ROOT SUCCESS (57)`, `FIREWALL_POLICY DISABLED (48)`, `SSH_CONFIG MODIFIED (46)`, `AUTHORIZED_KEY ADDED (40)`, and `AUDIT_CONFIG MODIFIED (31)`.
3. **Phase 2, about 12:40:00–12:41:00:** A flood of reassuring messages: `SYSTEM_HEALTH OK (112)`, `SECURITY_SCAN CLEAN (98)`, `FIREWALL_POLICY ENABLED (75)`, `INTEGRITY_CHECK PASS (71)`, `BACKUP COMPLETED`, and `SSH_SERVICE RUNNING`.

4.5 Likely objective

The attacker's goal was to poison the SOC's monitoring record. Phase 1 attributes root access, SSH-key persistence, and disabled firewall and audit controls to the trusted workstation. Phase 2 then buries those alerts under false assurance that health, scans, integrity, and firewall are normal. This produces the "operating normally" picture management wants to see. Combined with overlapping sequence numbers, the result is an inconsistent, untrustworthy log that hides the real state and misattributes activity to `workstation01`.

Limitation: A network capture proves the messages were forged. It cannot show whether the reported changes actually occurred. `workstation01`'s `authorized_keys`, SSH configuration, firewall, and audit settings should be checked directly, and `10.0.2.8` should be isolated and investigated.

Mitigation: Authenticate telemetry (TLS with client certificates or signed messages), alert on IP-to-MAC changes and duplicate sequence numbers, and enable ARP inspection or port security on the network.

5. Submitted Files

- `spoof_icmp.py`: Task 3 spoofed ICMP sender.
- `analyze_incident.py`: Task 4 PCAP feature extractor and anomaly summary.
- `incident_packets.csv`: extracted packet features from `lab1_incident.pcapng`.

Reproduce the Task 4 analysis: `python analyze_incident.py lab1_incident.pcapng --output incident_packets.csv`